

Ubuntu Bridge Initiative · Cybersecurity Internship · Cohort 1 · Stage 2 Capstone

Editable template — Deliverable 3 of 4. Target length: 3–5 pages. This is the heaviest deliverable — give it a full day. Two audiences, one document: a formal finding a technical lead can act on (Part 1), then a plain-language brief a CISO can quote to the board (Part 2). Replace every `[BRACKETED]` prompt. Remove every prompt and italic guidance line before submission.

~~The~~ **75-word executive summary goes ON THE PLATFORM — not inside this file.**

When you submit on `/dashboard/reports/STAGE_2`, the form has two fields: the **Drive folder URL**, and a **75-word executive summary** typed into the form. The summary names: which vuln you wrote up in D3, the root cause in one line, the highest-risk impact, and your CVSS base score. Draft it last, once this deliverable is final. It does **not** go inside D1–D4.

Deliverable 3 — Penetration test finding + board brief

~~Before~~ **Before you start writing, check yourself:**

>

- Have you **picked one** vulnerability to write up formally — the **SQL injection** OR the **SSRF**? (Not both. Pick the one you have the cleanest proof-of-work for.)
- Do you have your **proof of work** for that vuln to hand: your own flag, your exact payload, and the verbatim first-failure response? Part 1 ends with these and they are checked.
- Can you build a **CVSS v3.1 vector** and defend each metric? A wrong vector is a visible, gradeable error.
- Can you state the **root cause** as a cause, not a symptom? "Input wasn't validated" is a symptom and loses points.
- For Part 2: can you tie impact to the **84,210 PII records** and the **IAM blast radius** in money-and-risk terms, with **no jargon**?
- If any answer is no, finish that first.

% % Page 1 — Title and the one-line finding % %

Stage 2 · D3 · [Your Full Name] · [UBI-2026-####]

Stage 2 Capstone · Deliverable 3

Penetration test finding + board brief

Prepared for: Sankofa Digital — Engineering lead (Part 1) and CISO / Board (Part 2) **Author:** [Your full name] · **Intern code:** [UBI-2026-####] · **Date:** [DD Month 2026] **Engagement:** Reproduction of "The Griot" intrusion · lab environment

Vulnerability written up: [SQL injection in `login.php` — CWE-89] **OR** [SSRF to the cloud metadata service — CWE-918]. *Delete the one you did not pick.*

PART 1 — The finding (for a technical lead)

Use these section headings, in this exact order. Graders look for them by name.

Title

[A precise, single-line title. *Example: "Unauthenticated SQL injection in the legacy-admin login permits authentication bypass and full user-table disclosure."* Name the vuln, the location, and the worst outcome.]

Severity — CVSS v3.1

Vector: [CVSS:3.1/AV:??/AC:??/PR:??/UI:??/S:??/C:??/I:??/A:??] **Base score:** [X.X (Critical / High / ...)]

[Then 3–5 lines **justifying the non-obvious metrics**. Do not just paste a vector — defend it. *Example reasoning for the SQLi: AV:N (reachable over the network), AC:L (no special conditions), PR:N (no auth needed — that's the whole point), UI:N, S:U (impact stays in the app's scope) — but if you write up the SSRF, argue S:C (scope changes — the web app's vulnerability yields cloud-account credentials, a different security authority), which is exactly why the SSRF often scores higher. State your C/I/A reasoning tied to the real data exposed.*]

Affected component

[Exact endpoint/parameter/feature and version context. For the SQLi: `POST /legacy-admin/login.php`, `username` parameter, the hand-rolled query in `01-legacy-admin-login.php` online since 2019. For the SSRF: the URL-fetching feature and the metadata endpoint it reached.]

Reproduction

[Numbered steps a technical lead can follow exactly. Include the payload and a `curl` invocation.]

1. [Step]

2. [Step]

`curl -i -X POST 'https://sankofa.internal/legacy-admin/login.php' \ --data '[your exact payload]'`

Expected: [the exact observable result — 302 to `dashboard.php`, session issued, etc.]

Impact

[What an attacker achieves — tied to the specific data and the chain. For the SQLi: `auth bypass !' users.php 312-row disclosure !'` foothold for the rest of the chain. For the SSRF: temporary IAM credentials !' read access to `customer_pii` (84,210 records). Quantify.]

Root cause

[**Cause, not symptom.** *Example (SQLi): "The data-access layer concatenates the raw ``username`` POST value directly into the SQL string — `...WHERE username = '' . $username . ''` (see `01-legacy-admin-login.php`` lines 64–67). The query is built by string concatenation rather than a parameterised/prepared statement; user input and query structure are never separated."* Do not write "input was not validated."]

Recommended fix — and the one that would NOT fix it

[The real fix. *Example: "Replace the concatenated query with a parameterised prepared statement (``mysql`/PDO bound parameters`), so ``username`` can never alter query structure."*]

Then the **seductive wrong answer**, in one sentence: *"What would NOT fix it: adding a WAF rule that blocks `` OR '1'='1`` — it filters one signature, not the class; `` OR 1=1-- -``, encodings, and UNION variants walk straight past it. Input blocklisting treats the symptom."* Naming the fix that fails is a top-band discriminator.]

References (quote each in full)

- **OWASP:** [e.g. A03:2021 — Injection]
- **CWE:** [e.g. CWE-89 — Improper Neutralization of Special Elements used in an SQL Command]
- **MITRE ATT&CK:** [one technique ID + name, e.g. T1190 — Exploit Public-Facing Application]

Evidence appendix

[Every artefact this finding rests on: 02-attacker-http-capture.txt line(s), 01-legacy-admin-login.php lines, your lab command output, timestamps. One line each.]

Retest plan

[How Sankofa confirms the fix worked. *Example: "Re-run the reproduction ``curl`` against the patched endpoint; expect ``Invalid credentials`` (the injection no longer authenticates) and a parameterised-query code-review sign-off. Add a regression test sending `` OR '1'='1`` that must fail closed."*]

Proof of work

[**Required and checked.** Your own flag for the chosen vuln, your exact payload, and the verbatim response to your **first failed attempt**. Paste them in code blocks. Missing or wrong proof-of-work values cap this deliverable.] Flag: [your flag] Working payload: [exact] First-failure response: [verbatim]

Before you start Part 2, confirm Part 1:

>

- [] All headings present in order: Title · Severity · Affected component · Reproduction · Impact · Root cause · Recommended fix (+ what would NOT fix) · References · Evidence appendix · Retest plan · Proof of work.
- [] CVSS vector is complete and each non-obvious metric is justified.
- [] Root cause is a cause, not a symptom.
- [] The "would NOT fix it" sentence is present.
- [] Proof-of-work flag + payload + first-failure are pasted and correct.

PART 2 — The board brief (one page, plain language)

Audience: a CISO who will quote you to the board. No jargon. No CVE numbers, no MITRE IDs, no CVSS vector in this section — those live in Part 1. If a non-technical director cannot follow a sentence, rewrite it.

What happened, in plain terms

[2–3 sentences. *Example shape: "An attacker walked into a forgotten internal admin tool that had been online and unmaintained since 2019, tricked it into letting them in without a password, and from there reached the keys to our customer database. They did not need to break encryption or steal an employee's laptop — they used the front door we left unlocked."*]

What was at risk — in money-and-risk terms

[Tie it to the real numbers. *Example: "84,210 customer records — names, emails, account numbers, balances — were one query away. Under the NDPA, a breach of this size is a reportable event with regulatory exposure, mandatory customer notification, and the reputational cost of telling 84,210 people their bank details were reachable. The cloud credentials the attacker obtained would have let them read that entire database, not just a sample."* Anchor every claim to a real artefact — the 05-exfil-sample.csv field shape, the 84,210 figure, the IAM blast radius.]

The one control I recommend funding

[Name **one** control. Then — this is the discriminator — name **the one Stage 2 log line or payload it WOULD have stopped, and one it would NOT.**

Example: "Fund the decommissioning (or, if it must live, the placement behind authenticated SSO + WAF + egress filtering) of the legacy-admin app. WOULD have stopped: the `02:15:31` `admin` OR `1`=1` login bypass — SSO in front means there is no app-local login to inject into. Would NOT have stopped: a valid employee with stolen SSO credentials; this control closes the unauthenticated front door, not credential theft, which needs MFA — a separate ask I am flagging, not funding here."

Pre-submission checklist for D3

- [] **First line** is Stage 2 · D3 · <Full Name> · <Intern Code>.
- [] **One** vuln chosen for the finding (SQLi **or** SSRF) — not both.
- [] **All Part 1 headings** present and in order.
- [] **CVSS v3.1 vector** complete and justified (Scope metric defended — S:C for the SSRF, S:U for the SQLi).
- [] **Root cause** is a cause, not a symptom.
- [] **"What would NOT fix it"** sentence present.
- [] **References** quote OWASP ID + CWE ID + one MITRE ATT&CK ID in full.
- [] **Proof of work** (flag + exact payload + verbatim first-failure) present and correct.
- [] **Part 2** is plain language — no CVSS/CVE/MITRE/jargon — and tied to the 84,210 records + IAM blast radius.
- [] **The recommended control** names the log line/payload it would AND wouldn't stop.
- [] **No decoy** (Redis, TRAN{not - here}) cited as load-bearing (this carries a " 4 here).
- [] **Every`[BRACKETED]` prompt and italic guidance line** removed.
- [] **Page count is 3–5.** Saved as **PDF** (DOCX accepted).
- [] **File named** STAGE2_D3_REPORT_<Surname>.pdf.

— UBI Programme Team